



My ML Projects – Complete Technical Explanation for Interviews

This document explains all 4 of my machine learning projects in depth – every design choice, model used, and the reasoning behind each decision. Written for confident explanation in interviews.



Project 1: Multi-Label Bangla Sentiment Analysis

File: `Multi-class-sentiment-Analysis.py`

Status: First Author – Manuscript submitted to MDPI Q1 Journal (*Big Data and Cognitive Computing*)



What the Project Does

This project performs **multi-label sentiment classification** on Bangla text. Unlike ordinary sentiment analysis (positive/negative), each text can simultaneously carry **multiple emotions** from 5 classes: **Happy, Sad, Angry, Fear, Surprise**.

Example: A single sentence about a historical tragedy can be *Sad*, *Angry*, and *Fear* at the same time.

This is a **multi-label** (not multi-class) problem – each label is treated as a separate binary classification task.



Step-by-Step Technical Pipeline

Step 1: Data Loading & Cleaning

CSV dataset with columns: Text (Bangla), Happy, Sad, Angry, Fear, Surprise (binary labels)

- Dropped rows with missing text or labels

- Removed duplicates
- Renamed 'Paragraph' → 'Text' for clarity

Why clean first? Missing labels in multi-label settings silently corrupt all 5 classifiers simultaneously.

Step 2: Exploratory Data Analysis (EDA)

- **Sentiment distribution bar chart** – checked class imbalance (e.g., Happy might dominate over Fear)
- **Bangla word count distribution** – plotted histogram to understand text length variance
- **Co-occurrence analysis** – computed $P(\text{Sad}|\text{Happy})$, $P(\text{Angry}|\text{Fear})$ etc. using conditional probability
- **Jaccard overlap matrix** – symmetric measure of pair overlap: $|A \cap B| / |A \cup B|$
- **Chi-squared test** – checked statistical independence between every sentiment pair
- **Co-occurrence network** – visualized pairs with $P > 0.3$ as edges in a graph (NetworkX)
- **Outlier detection** – used Z-score ($|z| > 3$) on word counts to flag unusually long/short texts

Why all this EDA? In multi-label NLP, understanding which sentiments co-occur together helps explain model confusion between labels (e.g., Angry-Sad overlaps heavily → harder to distinguish).

Step 3: Text Preprocessing (The Critical Part)

This is where I addressed the core linguistic challenge of **Bangla's agglutinative morphology**.

What is agglutinative morphology?

In Bangla, suffixes are added to a root word to create grammatically different but semantically related forms. The same root can generate 100+ surface forms. This causes:

- BoW and TF-IDF to create **huge sparse vocabularies**
- Word2Vec to learn **poor embeddings** for rare word forms
- Out-of-vocabulary (OOV) words to be silently ignored

My preprocessing pipeline (in order):

Raw Bangla Text

↓

Unicode NFC normalization (fixes inconsistent character encoding in Bangla Unicode)

↓

Remove URLs, hashtags, @mentions, ellipses

↓
Keep only Bangla characters: `regex [^\u0980-\u09FF\s]`
↓
Bangla Stemming (`bnltk BanglaStemmer`) – reduces word to approximate root
↓
Custom Bangla stop word removal (500+ curated stop words)
↓
SentencePiece BPE tokenization (`vocab_size=10000, char_coverage=0.9995`)

Why SentencePiece BPE? Byte Pair Encoding learns subword units from data. For agglutinative Bangla:

- Rare compound forms are split into known subword pieces
- No OOV problem – every character is encodable
- Consistent tokenization across train and test

Why custom stop words? Standard NLTK stop words are English. Bangla requires a domain-specific curated list (I built 500+ words covering particles, conjunctions, pronouns in Bangla).

Step 4: Sentiment Lexicon Construction

I built a **hand-curated Bangla sentiment lexicon** with 30 words per emotion class (150 total):

- **Happy** : আনন্দ, সুখ, খুশি, উল্লাস...
- **Sad** : দুঃখ, কষ্ট, বিষণ্ণ, হতাশা...
- **Angry** : রাগ, ক্রোধ, ক্রোধ, বিরক্ত...
- **Fear** : ভয়, আতঙ্ক, উদ্বেগ, ভীত...
- **Surprise** : আশ্চর্য, বিস্ময়, অবাক, চমক...

Use: Lexicon features were used to analyze co-occurrence patterns in EDA and as an optional feature layer.

Step 5: Feature Extraction – TWO Separate Strategies

Strategy A: Sparse Features (BoW / TF-IDF / Char / Combined)

Feature	Config	Why
BoW	<code>CountVectorizer, max_features=3000, ngram_range=(1,7), word-level</code>	Captures word presence and short phrases
TF-IDF	<code>TfidfVectorizer, max_features=3000, ngram_range=(1,7), word-level</code>	Down-weights common words, up-weights discriminative ones

Feature	Config	Why
Char n-grams	TfidfVectorizer, max_features=7000, ngram_range=(3,8), char-level	KEY for Bangla – captures morphological suffixes even without stemming
Combined	<code>scipy.sparse.hstack([BoW, TF-IDF, Char])</code>	Horizontal stacking of all 3 features

Why char n-grams are critical for Bangla: A character 3-gram like "ার" (a common suffix) appears in many morphological variants of the same word. Char-level features capture this morphological regularity that word-level features miss.

Why `hstack` ? Horizontal concatenation of sparse matrices creates a richer feature space. Each column in the combined matrix represents a feature from one of the three representations.

Scaling for SVM: Used `MaxAbsScaler` (scales by absolute maximum, preserves sparsity) instead of `StandardScaler` (which would densify sparse matrices).

Strategy B: Dense Embedding Features

Feature	Config	Why
Word2Vec	Skip-gram, vector_size=100, window=10, min_count=1, epochs=50	Captures semantic similarity; skip-gram better for rare words
FastText	Skip-gram, vector_size=100, min_n=2, max_n=4, epochs=60	Best for Bangla OOV – uses subword character n-grams

Weighted average embedding:

For each word: $\text{weight} = \log(1 + \text{TF-IDF_score})$

Weighted average = $\frac{\sum(\text{word_vector} \times \text{weight})}{\sum(\text{weights})}$

Why weighted? TF-IDF weight ensures sentiment-discriminative words (rare but important) contribute more to the document vector than filler words.

FastText advantage over Word2Vec: FastText trains on character n-grams, so even unseen words get reasonable embeddings via their character substructure. Critical for agglutinative Bangla where new word forms appear constantly.

Step 6: Feature Selection

For Logistic Regression and SVM, I applied **per-label `SelectKBest`** :

- **Chi-squared (χ^2)** for sparse features (BoW, TF-IDF, Char, Combined)
- **ANOVA F-score** for dense embedding features (Word2Vec, FastText)

Why per-label? In multi-label classification, the most informative features differ between emotions. The word "রাগ" (anger) is highly predictive for *Angry* but not for *Happy*. Per-label selection allows each binary classifier to use its own optimal feature subset.

Chi-squared formula:

$$\chi^2 = \sum (O - E)^2 / E$$

High χ^2 for feature *f* and label *y* → the word appears much more/less often in positive vs. negative examples → highly discriminative.

k values: k=2300 for Logistic Regression, k=5000 for SVM.

Step 7: Model Building with MultiOutputClassifier

Core strategy: Wrap each base classifier in `sklearn.multioutput.MultiOutputClassifier`.

This trains **one independent binary classifier per sentiment label**:

```
MultiOutputClassifier(RandomForestClassifier(...))
→ RF_Happy, RF_Sad, RF_Angry, RF_Fear, RF_Surprise (5 separate models)
```

Model	Key Hyperparameters	Why Used
Random Forest	n_estimators=100-300, max_depth=5-10, balanced_class_weight	Robust to noise, handles high-dimensional sparse features
XGBoost	max_depth=3-7, learning_rate=0.05-0.1, scale_pos_weight, early_stopping=50	Best performance on tabular/sparse data; <code>scale_pos_weight</code> handles class imbalance
LightGBM	num_leaves=31-63, bagging_fraction, lambda_l1/l2, early_stopping=100	Faster than XGBoost, leaf-wise tree growth better for imbalanced data
SVM (LinearSVC)	C=0.01-1, L1/L2 penalty, balanced class_weight	Excellent with high-dimensional TF-IDF sparse features; L1 gives sparsity
Logistic Regression	C=0.01-1, L1/L2, solver=liblinear, balanced class_weight	Interpretable, fast, strong baseline for NLP; liblinear supports L1

Hyperparameter tuning: `RandomizedSearchCV` with `cv=3`, `scoring='f1_weighted'` — n_iter varied by model complexity (10 for RF, 20 for LightGBM, 30 for SVM/LR).

Class imbalance handling: `class_weight='balanced'` and `scale_pos_weight = neg_count/pos_count` (capped at 1.4 to prevent over-penalizing majority class for *Angry*).

Step 8: Threshold Optimization (Critical Innovation)

Instead of using the default 0.5 threshold for all labels, I optimized per-label thresholds on the **validation set**.

```
thresholds = np.arange(0.05, 0.99, 0.01)
# For each threshold, compute a custom score:
if col == 'Angry':
    score = 0.5*F1 + 0.45*Precision + 0.05*Recall
elif col == 'Sad':
    score = 0.52*F1 + 0.48*Precision
elif col == 'Happy':
    score = 0.58*F1 + 0.4*Precision
else:
    score = 0.65*F1 + 0.3*Precision + 0.05*Recall
```

Why emotion-specific scoring?

- *Angry* has the most frequent false positives → emphasize Precision
 - *Happy* is the majority class → balance F1 and Precision
 - This is domain-specific threshold tuning based on label statistics
-

Step 9: Ensemble Methods

Voting (Soft Voting)

Trees (XGBoost + LightGBM + RandomForest) → average their probability outputs

SVM variants (SVM + LR) → average their sigmoid-transformed decision scores

Final prediction: threshold the averaged probability

Why soft voting? Averaging probabilities reduces individual model variance without sacrificing calibration.

Stacking

Base models → generate probability outputs (meta-features)

Meta-learner (Logistic Regression, C=0.1) → trained on stacked meta-features

Why LR as meta-learner? Lightweight, regularized ($C=0.1$), prevents meta-learner from overfitting on base model outputs.

Hybrid

Tree ensemble (XGB+LGB+RF) → average probabilities
Horizontally stack [SVM_features | tree_probabilities]
New LinearSVC ($C=0.01$) → trained on this augmented feature space

Why hybrid? The SVM gets both the rich sparse text features AND the probabilistic outputs of tree models – combining complementary representations.

Step 10: Evaluation Metrics

For multi-label classification:

Metric	Why Used
Hamming Loss	Fraction of labels predicted incorrectly across all samples and labels. Lower = better.
Subset Accuracy	Fraction of samples where ALL labels are correct. Strict.
F1-Macro	Unweighted average F1 across all 5 labels – treats all classes equally
F1-Micro	Aggregate TP/FP/FN across all labels – influenced by majority class
Jaccard Score (Micro)	,
ROC-AUC per label	Ranking ability of each per-label binary classifier

 How to Explain This in Interview

"My Bangla sentiment analysis project tackles multi-label classification across 5 emotion categories simultaneously. The core challenge was Bangla's agglutinative morphology – a single root word produces hundreds of surface forms, breaking standard tokenizers. I addressed this with a layered approach: Unicode normalization, Bangla-specific stemming using `bnltk`, custom stop word removal, followed by `SentencePiece` BPE tokenization. For features, I ran two parallel strategies: sparse features using BoW, TF-IDF, and character n -grams horizontally stacked, and dense embedding features using TF-IDF-weighted Word2Vec and FastText. FastText was particularly important for OOV handling via subword character n -grams. I used chi-squared and ANOVA F for per-label feature selection, then trained 5 models – RF, XGBoost, LightGBM, SVM, Logistic Regression – with `RandomizedSearchCV` tuning. I also optimized decision thresholds per

emotion label on the validation set, with emotion-specific scoring formulas. Finally, I combined models using voting, stacking, and a hybrid tree-to-SVM approach."

Project 2: Bangla Sign Language Detection (EfficientNet + Attention)

File: `efficientnet_b1_b4.py`

Purpose: Real-time Bangla Sign Language (BdSL) recognition for hearing-impaired community

What the Project Does

Classifies **48 Bangla sign language hand gestures** from images/webcam in real-time, using EfficientNet (B1-B4) with a **custom Spatial Attention module**, trained via **transfer learning** and explained via **Grad-CAM**.

Step-by-Step Technical Pipeline

Step 1: Data Preparation & Augmentation

Dataset structure: `ImageFolder` format with `train/val/test` splits and 48 class folders (one per Bangla sign).

Training augmentations (why each one):

```
transforms.RandomRotation(20)           # Signs at slight angles
transforms.RandomHorizontalFlip(0.5)     # Mirrored hands
transforms.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4, hue=
# Handles lighting variation (indoor/outdoor, different skin tones)
transforms.RandomErasing(p=0.5, scale=(0.02, 0.2))
# Occlusion augmentation - simulates partial hand visibility
transforms.Normalize([0.485,0.456,0.406], [0.229,0.224,0.225])
# ImageNet statistics - required because backbone was pre-trained on ImageNet
```

Why ImageNet normalization? EfficientNet was pre-trained on ImageNet. Using the same mean and std ensures input distribution matches what the pre-trained layers expect, making transfer learning

effective.

DataLoader config: `batch_size=48` , `pin_memory=True` (faster GPU transfer),
`num_workers=2` .

Step 2: Transfer Learning with EfficientNet

Why EfficientNet? EfficientNet scales model width, depth, and resolution uniformly using a compound scaling coefficient. Compared to ResNet:

- Better accuracy per parameter
- More efficient computation
- B1-B4 variants offer progressively larger capacity

Fine-tuning strategy:

```
# Freeze early layers (learn low-level features like edges)
# Unfreeze LAST 3 blocks only
for layer in self.features[-3:]:
    for p in layer.parameters():
        p.requires_grad = True
```

Why unfreeze only last 3 blocks?

- Early layers learn universal features (edges, textures) that transfer well
- Later layers learn task-specific features (hand shape, finger positions)
- Unfreezing only later blocks prevents catastrophic forgetting while allowing task adaptation

Classifier head:

```
nn.Dropout(0.4)          # Regularization — prevents overfitting on small data
nn.Linear(in_features, num_classes)  # 48-class output
```

Step 3: Spatial Attention Module

```
class SpatialAttention(nn.Module):
    def forward(self, x):
        avg_out = torch.mean(x, dim=1, keepdim=True)  # Channel-wise average
        max_out, _ = torch.max(x, dim=1, keepdim=True) # Channel-wise max
        x_cat = torch.cat([avg_out, max_out], dim=1)   # Concatenate [2,
```

```
x = self.conv(x_cat)    # 2→1 channel 7×7 conv
return self.sigmoid(x) # Attention map in [0,1]
```

Applied as: `x = features * attention_map` (element-wise multiplication)

Why Spatial Attention?

- Sign language gestures have spatially localized information (hand shape, finger joints)
- Background pixels are noise – attention suppresses them
- The model learns WHERE to look, not just what features matter
- Inspired by CBAM (Convolutional Block Attention Module)

Why avg + max pooling? Average captures overall channel importance; max captures the most salient activation. Together they provide complementary spatial context.

Experiment design: Trained TWO models – `EfficientNet_Attention` and `EfficientNet_NoAttention` – to ablate the contribution of the spatial attention module.

Step 4: Training with ReduceLROnPlateau

```
optimizer = optim.Adam(model.parameters(), lr=5e-4, weight_decay=1e-5)
scheduler = lr_scheduler.ReduceLROnPlateau(optimizer, mode='max', factor=
```

Why Adam? Adaptive learning rates per parameter – handles sparse gradients and different parameter scales well. `weight_decay=1e-5` = L2 regularization to prevent overfitting.

Why ReduceLROnPlateau? Monitors validation accuracy; if it doesn't improve for 5 epochs, halves the learning rate. This allows aggressive early learning then fine refinement – avoids manual learning rate scheduling.

Loss: `CrossEntropyLoss` – standard for multi-class classification. Internally applies softmax.

Best model saving: Saved the checkpoint with highest validation accuracy using `torch.save(model.state_dict(), ...)`.

Step 5: Evaluation

```
def evaluate(model):
    y_true, y_pred = [], []
    with torch.no_grad():          # Disable gradient computation for speed
```

```
for images, labels in test_loader:
    outputs = model(images)
    _, preds = torch.max(outputs, 1) # argmax over 48 classes
```

- `classification_report` – per-class Precision, Recall, F1, Support
 - Confusion matrix (28×24 inch) with Noto Bengali font for Bangla class labels
-

Step 6: Grad-CAM Visualization

What is Grad-CAM? Gradient-weighted Class Activation Mapping – visualizes which spatial regions the CNN focused on for a prediction.

Forward pass → record activations of last conv layer

Backward pass on predicted class score → record gradients

`weights = GlobalAveragePool(gradients)` # Importance of each feature map

`CAM = ReLU( $\sum$  weights × activations)` # Weighted activation sum, clipped positive

Resize to input resolution → overlay on image

Specific enhancements I made:

- Large dilation kernel (17×17, 4 iterations) to expand the heatmap to cover full hand + all fingers
- Suppress background (threshold at 12th percentile)
- Show: Original → Raw CAM → Normalized CAM → Overlay (4 columns per class)

Why Grad-CAM? Interpretability. In a medical/accessibility application (blind/hearing-impaired users), it's critical to verify the model focuses on the hand gesture, not background artifacts.

Step 7: Real-Time Inference

```
# Center crop 80% of frame → assumes hand fills center
size = int(0.8 * min(h, w))
crop = frame_rgb[center_y-size//2 : center_y+size//2,
                 center_x-size//2 : center_x+size//2]
```

- **No MediaPipe** – removed dependency to eliminate "no hand detected" failures
 - Inference via `F.softmax(output, dim=1)` → confidence score
 - Color-coded text: Green (>80%), Orange (50-80%), Red (<50%)
-

How to Explain in Interview

"The BdSL project classifies 48 Bangla sign language gestures using EfficientNet with a custom Spatial Attention module. I used transfer learning – loading ImageNet pre-trained EfficientNet B1 through B4 weights, then fine-tuning only the last 3 blocks while freezing the rest. The key contribution was the spatial attention mechanism: it computes channel-wise average and max pooling, concatenates them, applies a 7×7 convolution to produce a spatial attention map, and multiplies it with the feature map. This helps the model focus on hand regions and suppress background noise. I ran an ablation study comparing models with and without attention. For interpretability, I implemented Grad-CAM with aggressive dilation kernels to visualize full hand coverage. Real-time inference uses a center-crop strategy at ~4 FPS without needing MediaPipe."

Project 3: Chair Detection with Privacy-Aware Guidance for Blind Users

File: `chair_detect_best_privacy.py`

Purpose: Help visually impaired people find the best available, privacy-respecting empty seat

What the Project Does

A real-time computer vision system that:

1. **Detects chairs** and determines if they are occupied or empty
 2. **Estimates distance** to each chair using monocular depth estimation
 3. **Scores chairs** based on distance AND privacy (people nearby)
 4. **Guides the user** to the best chair via voice, with a lock-and-navigate system
 5. **Tracks chairs** persistently across frames using Kalman Filters
-

Component-by-Component Breakdown

Component 1: Object Detection – YOLOv8 Large Segmentation

```
model_yolo = YOLO('yolov8l-seg.pt') # Large segmentation variant
results = model_yolo(frame, verbose=False, conf=0.20)
```

Why YOLOv8l-seg?

- **l** = large model → better accuracy, especially for **partial chair views** (side, back, angled)
- **-seg** = segmentation variant → produces **per-instance pixel masks** in addition to bounding boxes
- Masks are used for accurate depth estimation (sample only chair pixels, not background)
- Low confidence threshold (0.20) to catch partially visible chairs

COCO classes used: Class 56 = chair, Class 0 = person

Component 2: Occupancy Detection

Primary method – IoU (Intersection over Union):

```
effective_iou = calculate_iou(chair_bbox, person['bbox'])
overlap_score = effective_iou * (person_conf + chair_conf) / 2
if max_overlap > 0.22:
    chair['occupied'] = True
```

Why bounding box IoU for occupancy?

- Works for ALL viewing angles (front, back, side) – unlike seat-area masks which need front-facing views
- Confidence-weighted overlap prevents false positives from low-confidence detections
- Threshold 0.22 was chosen to handle partial overlap (someone leaning over a chair)

Secondary confirmation – MediaPipe Pose (hip depth):

```
hips_avg_z = (left_hip.z + right_hip.z) / 2
if hips_avg_z < -0.05: # Hips pushed forward = sitting posture
    chair['occupied'] = True
```

Why hip z-coordinate? In MediaPipe's normalized coordinate system, when a person sits, their hips move forward (negative z direction) relative to the camera. This is a biomechanical marker of sitting.

Two-stage logic: IoU determines primary occupancy; pose acts as confirmation for borderline cases ($0.15 < \text{overlap} < 0.22$).

Component 3: Distance Estimation – MiDaS DPT_Large

```

midas = torch.hub.load('intel-isl/MiDaS', 'DPT_Large')
# ... run inference ...
depth_map = prediction.cpu().numpy()

def calibrate(raw):
    return max(0.5, 22.0 / (raw + 14.0))

```

Why MiDaS? Monocular depth estimation from a single RGB image – no stereo camera or LiDAR needed. DPT_Large (Dense Prediction Transformer) is the most accurate variant.

Why sample the lower 40% of the chair mask for depth?

```

seat_start = int(0.6 * h) # Bottom 40% of bounding box
valid = depth_crop[seat_start:, :][valid_mask]

```

The seat is in the lower portion of the chair bounding box. Sampling only the seat area avoids backrest/legs which are at different depths.

Calibration function $22.0 / (raw + 14.0)$: MiDaS outputs relative disparity (inverse depth). This empirical formula converts to approximate metric meters. $\max(0.5, \dots)$ prevents distance of 0.

Component 4: Chair Scoring for Best Chair Selection

```

chair['best_score'] = (chair['distance'] * 0.55) + (chair['nearby_persons'] * 0.45)
empty_chairs.sort(key=lambda x: x['best_score'])
best_chair = empty_chairs[0] # Lowest score = best

```

Scoring formula explained:

- **Distance weight (0.55):** Closest chair is preferred (user walks less)
- **Nearby persons weight (0.45):** Chair with fewer people nearby = more privacy

Privacy radius: 180 pixels. Any person detected within 180px of the chair center counts as "nearby".

Why this specific 55/45 split? The weights were chosen to balance physical accessibility (distance) with social comfort (privacy). For a blind user, privacy from strangers is almost as important as proximity.

Privacy message tiers:

- 0 people nearby → "Excellent privacy – no one nearby"

- 1-2 people → "Good privacy"
 - 3+ people → "Best available for privacy" (still guide, just acknowledge)
-

Component 5: Kalman Filter Tracking

```
class ChairTracker:
    # State vector: [cx, cy, vx, vy] (position + velocity)
    kf.F = np.array([[1,0,1,0],[0,1,0,1],[0,0,1,0],[0,0,0,1]]) # Transit
    kf.H = np.array([[1,0,0,0],[0,1,0,0]]) # Observation matrix
    kf.R = 8 # Measurement noise
    kf.Q = eye(4)*0.1 # Process noise
```

Why Kalman Filter for tracking?

- Chairs may be temporarily occluded (person walks in front) – KF predicts position during occlusion
- Assigns stable `track_id` to each chair across frames
- Enables the lock-and-navigate system to follow the same physical chair

Matching strategy: Euclidean distance between KF predicted center and detected center. If distance < 60px → same chair; otherwise → new chair.

State removal: When the KF uncertainty (`P[0,0] > 15000`) exceeds threshold → chair likely left the scene.

Component 6: Lock & Navigate System

Lock state machine:

```
SEARCHING → [User says YES to confirmation] → LOCKED
LOCKED    → [navigate_to_locked()] → guides user each frame
LOCKED    → [distance < 1.2m] → ask_sit_confirmation()
CONFIRMED → [User says YES, sat] → reset LockState → SEARCHING
```

Re-anchoring across camera movement:

```
# Fast path: match by track_id (stable when camera is still)
# Slow path: match by minimum center distance < 200px
#             (works when camera moves and tracker reassigns IDs)
```

This dual-path strategy prevents losing the locked chair when the user turns their head.

Navigation guidance:

```
angle = (center_x - frame_center) / frame_width * fov_deg # FOV = 78°  
direction = "left" if angle < -10 else "right" if angle > 10 else "straig
```

At $> 40^\circ$ deviation $\rightarrow$ "Wrong direction, turn [direction]"

At $< 10^\circ$ $\rightarrow$ "Straight ahead, X meters"

Else $\rightarrow$ "Turn [direction] about X degrees"

Voice output: gTTS (Google Text-to-Speech) $\rightarrow$ save MP3 $\rightarrow$ display as audio in Colab $\rightarrow$

`time.sleep` calibrated to speech length.

User confirmation UI: JavaScript overlay (YES/NO buttons) + speech recognition fallback – both polling

```
eval_js('window.__chairLockAnswer') .
```

Component 7: Ghost Box for Out-of-Frame Chair

```
if lock_state.frames_out_of_view > 0 and lock_state.last_bbox is not None  
    # Draw dashed blue rectangle at last known position  
    # Label: "LOCKED (out of frame - go [side])"
```

When the locked chair exits the camera frame, the system draws its last known bounding box as a **ghost** with directional guidance – preventing the user from panicking when the chair temporarily disappears.

How to Explain in Interview

"The chair detection project is a real-time assistive computer vision system for blind users. It uses YOLOv8l-seg for simultaneous chair and person detection with pixel masks. Occupancy is determined by bounding box IoU between chair and person detections, with MediaPipe pose as a secondary confirmation using hip z-coordinates as a sitting biomechanical marker. Distance is estimated with MiDaS monocular depth, sampling only the seat area of the chair mask. Each empty chair is scored on a weighted combination of distance and privacy – how many people are nearby. Kalman filters provide stable chair tracking across frames, enabling the lock-and-navigate system where the user confirms a target chair and is guided there via gTTS voice feedback with angle and distance instructions."



Project 4: Spam Detection

File: `spam_detection_final.py`

Purpose: Binary email/SMS classification (Ham vs. Spam) – end-to-end NLP pipeline



What the Project Does

Classifies SMS/emails as spam (1) or ham (0) using classical NLP preprocessing and Naive Bayes classifiers, with extended comparison across 8 models including ensemble methods.



Step-by-Step Technical Breakdown

Step 1: Data Cleaning

```
df.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], inplace=True)
df.rename(columns={'v1': 'target', 'v2': 'text'}, inplace=True)
encoder = LabelEncoder()
df['target'] = encoder.fit_transform(df['target']) # ham=0, spam=1
df = df.drop_duplicates(keep='first')
```

- Removed 3 empty artifact columns from the SMS Spam Collection dataset
 - Label encoding: `ham` → 0, `spam` → 1
 - Dropped 415 duplicate messages (important for evaluation integrity)
-

Step 2: EDA

```
df['num_characters'] = df['text'].apply(len)
df['num_words'] = df['text'].apply(lambda x: len(nltk.word_tokenize(x)))
df['num_sentences'] = df['text'].apply(lambda x: len(nltk.sent_tokenize(x)))
```

Key findings from EDA:

- Dataset is **imbalanced**: ~86.6% ham, ~13.4% spam
- Spam messages are notably longer (more characters and words)
- Spam has more sentences (multi-offer messages)

- Histogram and pairplot confirm these distributions differ significantly

WordCloud analysis: Generated spam and ham word clouds to visually identify distinguishing vocabulary (spam: "free", "win", "call", "prize"; ham: personal pronouns, casual words).

Step 3: Text Preprocessing Pipeline

```
def transform_text(text):
    text = text.lower()                # Lowercase
    text = nltk.word_tokenize(text)    # Tokenize
    # Keep only alphanumeric tokens
    y = [i for i in text if i.isalnum()]
    # Remove stop words and punctuation
    y = [i for i in y if i not in stopwords.words('english')
          and i not in string.punctuation]
    # Apply Porter Stemmer
    y = [ps.stem(i) for i in y]
    return " ".join(y)
```

Why Porter Stemmer?

- English spam text uses many word forms: "winning/winner/win" → "win"
- Stemming reduces vocabulary size → less sparse TF-IDF matrix
- Trade-off: may produce non-words but improves generalization

Why remove stop words?

- "the", "is", "a" carry no spam/ham signal
 - Reduces feature dimensionality
 - TF-IDF would down-weight them anyway, but removing explicitly is cleaner
-

Step 4: Feature Vectorization – TF-IDF

```
tfidf = TfidfVectorizer(max_features=3000)
X = tfidf.fit_transform(df['transformed_text']).toarray()
```

Why TF-IDF over BoW for spam?

- Words like "free" appear in many spam messages → high TF but also high DF
- TF-IDF penalizes words that appear everywhere → focuses on discriminative terms

- `max_features=3000` limits vocabulary to top 3000 terms by TF-IDF score

Why `max_features=3000` ? The full vocabulary after preprocessing has thousands of unique stems. Limiting to 3000 prevents overfitting and reduces memory.

Step 5: Primary Model – Naive Bayes Variants

Three Naive Bayes variants were compared:

Model	Assumption	Best For
GaussianNB	Features follow Gaussian distribution	Continuous features
MultinomialNB	Features are non-negative counts/frequencies	TF-IDF (best match)
BernoulliNB	Features are binary (present/absent)	Binary BoW features

Why MultinomialNB wins for TF-IDF:

TF-IDF values are non-negative real numbers representing term frequency × inverse document frequency. MultinomialNB treats them as event probabilities proportional to counts. It's designed for this type of discrete count/frequency data.

Bayes theorem for text classification:

$$P(\text{spam} \mid \text{text}) \propto \frac{P(\text{spam}) \times \prod P(\text{word}_i \mid \text{spam})}{P(\text{text})}$$

"Naive" assumption: words are conditionally independent given the class.

Step 6: Extended Model Comparison (8 Models)

```
clfs = {
    'SVC': SVC(kernel='sigmoid', gamma=1.0),
    'KN': KNeighborsClassifier(),
    'NB': MultinomialNB(),
    'DT': DecisionTreeClassifier(max_depth=5),
    'LR': LogisticRegression(solver='liblinear', penalty='l1'),
    'RF': RandomForestClassifier(n_estimators=50),
    'AdaBoost': AdaBoostClassifier(n_estimators=50),
    'BgC': BaggingClassifier(n_estimators=50)
}
```

Why sigmoid kernel SVM for spam? Text classification often has non-linear boundaries. Sigmoid kernel approximates neural network activation.

Why `max_depth=5` for Decision Tree? Prevents overfitting on high-dimensional TF-IDF features.

Why L1 penalty for Logistic Regression? Spam detection involves many irrelevant features. L1 drives coefficients for non-spam-related words to exactly 0 – automatic feature selection.

Evaluation metric: Precision was prioritized over accuracy because:

- The dataset is imbalanced (86% ham)
 - A model that classifies everything as ham achieves 86% accuracy but 0 spam recall
 - High precision = when we say "spam", we're right → minimizes legitimate emails being filtered
-

Step 7: Ensemble Methods

Voting Classifier (Soft Voting):

```
voting = VotingClassifier(  
    estimators=[('svm', svc), ('nb', mnb), ('rf', rfc)],  
    voting='soft' # Average probabilities, not hard labels  
)
```

Why soft voting? SVM with `probability=True` gives calibrated probabilities. Soft voting lets the model confidence influence the final decision.

Stacking Classifier:

```
estimators = [('svm', svc), ('nb', mnb), ('rf', rfc)]  
final_estimator = RandomForestClassifier()  
clf = StackingClassifier(estimators=estimators, final_estimator=final_estimator)
```

Why Random Forest as meta-learner? RF is non-linear and handles correlated meta-features well. The base models (SVM, NB, RF) may make correlated errors – RF captures complex interactions.

Step 8: Visualization & Reporting

- **Confusion Matrix** (via `ConfusionMatrixDisplay`) – shows TP, FP, FN, TN for spam/ham
- **ROC-AUC curves** for all 3 Naive Bayes variants – visualizes threshold-independent discriminative ability
- **Grouped bar charts** – Accuracy and Precision side-by-side across models

How to Explain in Interview

"The spam detection project is a classic NLP binary classification pipeline on the SMS Spam Collection dataset. The preprocessing chain – lowercase, tokenize, alphanumeric filter, stop word removal, Porter stemming – converts raw SMS text into a clean stemmed representation. TF-IDF with 3000 features creates the feature matrix. I compared three Naive Bayes variants: Gaussian, Multinomial, and Bernoulli. MultinomialNB performs best with TF-IDF because TF-IDF values are non-negative frequency-like features that match the multinomial distribution assumption. I extended the comparison to 8 classifiers and implemented both soft voting and stacking ensembles. I prioritized Precision as the key metric because spam detection errors that filter legitimate emails are more costly than missing some spam."

Quick Reference: All Projects at a Glance

Project	Task Type	Key Models	Key Technique	Unique Contribution
Bangla Sentiment	Multi-label NLP	RF, XGB, LGB, SVM, LR	SentencePiece BPE, char n-grams, per-label threshold optimization	Agglutinative Bangla handling via subword tokenization + TF-IDF-weighted embeddings
Bangla Sign Language	Image Classification	EfficientNet B1-B4	Transfer Learning, Spatial Attention, Grad-CAM	Spatial attention ablation study; Grad-CAM with dilation for full hand coverage
Chair Detection	Object Detection + Navigation	YOLOv8l-seg, MiDaS, MediaPipe	IoU occupancy, KF tracking, monocular depth	Privacy-aware scoring (distance + nearby persons) + lock-and-navigate voice guidance
Spam Detection	Binary NLP Classification	MultinomialNB, SVM, RF, Ensemble	TF-IDF, Porter Stemmer, Soft Voting, Stacking	Full 8-model comparison with ensemble study

Common Interview Questions About These Projects

Q: What was the hardest technical challenge?

"In the Bangla sentiment project, handling agglutinative morphology was the core challenge. A single root word produces hundreds of surface forms that appear as distinct vocabulary items. I combined BPE subword tokenization (which handles unseen forms), character n-gram TF-IDF (which captures morphological patterns), and TF-IDF-weighted FastText (which handles OOV via subword character embeddings). No single approach solved it – the combination did."

Q: How did you handle class imbalance?

"In spam detection – `class_weight='balanced'` and prioritizing Precision over Accuracy. In Bangla sentiment – `class_weight='balanced'` for all classifiers, `scale_pos_weight = neg/pos` for XGBoost/LightGBM (capped at 1.4), and per-label threshold optimization on the validation set."

Q: Why did you choose multi-label over multi-class for sentiment?

"Human emotions co-occur naturally. A text about historical injustice can genuinely be Angry, Sad, and Fear simultaneously. Forcing single-label classification loses this information. My co-occurrence analysis showed pair overlaps above 0.3 probability for most emotion pairs, confirming that multi-label is the correct problem formulation."

Q: How does Grad-CAM work?

"Grad-CAM hooks into the last convolutional layer's forward pass to capture feature map activations, and backward pass to capture gradients w.r.t. the predicted class score. Weights are the global average of these gradients. The CAM is the ReLU of the weighted sum of activations, resized to input resolution, and overlaid as a heatmap."

Q: What would you improve?

"For Bangla sentiment: fine-tuning BanglaBERT (a transformer pre-trained on Bangla text) would likely outperform all sparse and static embedding approaches. For chair detection: a stereo camera would give true metric depth instead of the calibrated MiDaS estimate. For sign language: adding temporal modeling (LSTM over frame sequences) would improve continuous sign recognition beyond isolated gesture classification."